

CO1-AT3-Coding Challenge

View Only

[← Back](#)

QUESTIONS

Q1

Smart University Payroll
Management System
25 marks

Q2

Smart Vehicle Insurance Claim
System
25 marks

2/2 answered

Q2 Smart Vehicle Insurance Claim System

25 marks

Problem Statement An insurance company processes claims for three categories of vehicles: Car, Bike, and Truck. The claim amount depends on the vehicle type. **Requirements** Superclass: Vehicle Private members: vehicleNumber ownerName insuredAmount Subclasses: Car Bike Truck Each subclass overrides calculateClaimAmount() using different formulas. The application should: Read details for N vehicles Store them using an array of superclass references Calculate claim amounts using runtime polymorphism Search by vehicle number Display the vehicle with the highest claim amount

Question 3 (Very Hard) Online Shopping Order Processing System (50 Marks) **Problem Statement** An e-commerce company sells products in three categories: Electronics Clothing Grocery The billing process varies for each category. **Requirements** Superclass: Product Private members: productId productName price quantity Subclasses: Electronics Clothing Grocery Each subclass overrides calculateBill(). The program should: Store products in an array of superclass references Calculate the final bill using runtime polymorphism Search products by ID Display the most expensive order Compute total revenue

Claim Formula (Recommended)

Car

$$\text{Claim} = \text{insuredAmount} \times 0.80$$

Bike

Claim = insuredAmount × 0.60

Truck

Claim = insuredAmount × 0.90

Test Case 1

Input

3

Car

TN10AB1234

Ramesh

800000

Bike

TN09XY5678

Suresh

100000

Truck

TN11TR9999

Logan

1500000

Search Vehicle:

TN09XY5678

Expected Output

Vehicle : TN10AB1234

Claim Amount : 640000.0

Vehicle : TN09XY5678
Claim Amount : 60000.0

Vehicle : TN11TR9999
Claim Amount : 1350000.0

Highest Claim Vehicle
TN11TR9999
1350000.0

Vehicle Found
Owner : Suresh
Claim Amount : 60000.0

Your Code

```
1 import java.util.Scanner;  
2 class Vehicle  
3 {  
4     private String vehicleNumber;  
5     private String ownerName;  
6     private double insuredAmount;  
7     Vehicle(String vehicleNumber, String ownerName,  
8         {  
9         this.vehicleNumber = vehicleNumber;  
10        this.ownerName = ownerName;  
11        this.insuredAmount = insuredAmount;  
12    }  
13    public String getVehicleNumber()  
14    {  
15        return vehicleNumber;  
16    }
```

Input

3
Car
TN10AB1234
Ramesh

Output

Vehicle: TN10AB1234
Claim Amount: 640000.0

Vehicle: TN09XY5678
Claim Amount: 60000.0

```
17     public String getOwnerName()
18     {
19         return ownerName;
20     }
21     public double getInsuredAmount()
22     {
23         return insuredAmount;
24     }
25     public double calculateClaimAmount()
26     {
27         return 0.0;
28     }
29     public static void main(String args[])
30     {
31         Scanner sc = new Scanner(System.in);
32         int n = Integer.parseInt(sc.nextLine());
33         Vehicle vehicles[] = new Vehicle[n];
34         for(int i=0;i<n;i++)
35         {
36             String type = sc.nextLine();
37             String vehicleNumber = sc.nextLine();
38             String ownerName = sc.nextLine();
39             double insuredAmount = Double.parseDouble(sc.nextLine());
40             if(type.equalsIgnoreCase("Car"))
41             {
42                 vehicles[i] = new Car(vehicleNumber, ownerName, insuredAmount);
43             }
44             else if(type.equalsIgnoreCase("Bike"))
45             {
46                 vehicles[i] = new Bike(vehicleNumber, ownerName, insuredAmount);
47             }
48             else if(type.equalsIgnoreCase("Truck"))
49             {
```

```

50         vehicles[i] = new Truck(vehicleNumk
51     }
52 }
53 String searchVehicle = sc.nextLine();
54 Vehicle highest = vehicles[0];
55 for(int i=0;i<n;i++)
56 {
57     System.out.println("Vehicle: " + vehicl
58     System.out.println("Claim Amount: " + \
59     System.out.println();
60     if(vehicles[i].calculateClaimAmount() >
61     {
62         highest = vehicles[i];
63     }
64 }
65 System.out.println("Highest Claim Vehicle")
66 System.out.println(highest.getVehicleNumber
67 System.out.println(highest.calculateClaimAr
68 System.out.println();
69 boolean found = false;
70 for(int i=0;i<n;i++)
71 {
72     if(vehicles[i].getVehicleNumber().equal
73     {
74         found = true;
75         System.out.println("Vehicle Found")
76         System.out.println("Owner: " + vehi
77         System.out.println("Claim Amount: '
78         break;
79     }
80 }
81 if(!found)
82 {

```

```
83         System.out.println("Vehicle Not Found")
84     }
85     sc.close();
86 }
87 }
88 class Car extends Vehicle
89 {
90     Car(String vehicleNumber,String ownerName,doubl
91     {
92         super(vehicleNumber,ownerName,insuredAmount
93     }
94     public double calculateClaimAmount()
95     {
96         return getInsuredAmount()*0.80;
97     }
98 }
99 class Bike extends Vehicle
100 {
101     Bike(String vehicleNumber,String ownerName,douk
102     {
103         super(vehicleNumber,ownerName,insuredAmount
104     }
105     public double calculateClaimAmount()
106     {
107         return getInsuredAmount()*0.60;
108     }
109 }
110 class Truck extends Vehicle
111 {
112     Truck(String vehicleNumber,String ownerName,dou
113     {
114         super(vehicleNumber,ownerName,insuredAmount
115     }
```

```
116     public double calculateClaimAmount()  
117     {  
118         return getInsuredAmount()*0.90;  
119     }  
120 }
```

